

Clinical & Coding Validation System

Knowledge Transfer Document

Version: 0.1.0 | **Date:** September 2026

Status: Active Development

Version: 0.1.0 **Date:** September 2026 **Status:** Active Development

Table of Contents

Part I — Start Here: Plain-English Guide

- I.1 What Is Coding Validation?
- I.2 What Is Clinical Validation?
- I.3 Why We Need Both
- I.4 Why AI Alone Is Not Enough
- I.5 One Case, Walked Through End to End
- I.6 How the APIs Work, Step by Step

Part II — Technical Reference

1. Project Overview & Scope
2. Solution Architecture
3. Data Model & Case Schema
4. Ingestion & Case Parsing Layer
5. Knowledge Graph & Guideline Retrieval
6. AI Integration & Prompt Engineering
7. Deterministic Clinical Validation Engine

- 8. Compliance Rules Engine
- 9. Audit Logging & Tamper Evidence
- 10. NLP Extraction Adapter
- 11. SDK Extraction & Reusable Package
- 12. Infrastructure & DevOps
- 13. Testing Strategy
- 14. API Reference
- 15. Configuration & Environment
- 16. Future Enhancements
- 17. Appendix A: File Structure Map
- 18. Appendix B: Data Flow Diagrams
- 19. Appendix C: Key Configuration Reference

Part I — Start Here: The System in Plain English

New to medical coding or clinical validation? Read this part first — it uses everyday language and concrete examples. The detailed technical reference begins at Section 1.

1.1 What Is Coding Validation?

Every time a patient leaves a hospital, the hospital sends an insurance claim. To get paid, it must describe the visit using standardized codes rather than free text. Three kinds of codes matter most:

Code type	What it describes	Example
ICD-10-CM diagnosis code	The conditions the patient had	U07.1 = COVID-19
ICD-10-PCS procedure code	The procedures performed	0BH17EZ = inserting a breathing tube
DRG (Diagnosis-Related Group)	The payment bucket — decided by the diagnosis, procedures, and complications	DRG 177 = "Respiratory Infections and Inflammations with MCC"

Coding validation is the process of checking that these codes are correct. "Correct" means three things:

- Supported by the documentation** — the clinical note actually describes the condition and treatment being billed.
- Chosen and sequenced by the official rules** — the *ICD-10-CM Official Guidelines for Coding and Reporting* say which condition must be listed first, which codes may or may not appear together, and how complications change the DRG.

3. **Consistent across the claim** — the DRG matches the diagnoses and procedures, and the Present-on-Admission (POA) flags are right.

A concrete example

A 62-year-old is admitted with COVID-19 and pneumonia and needs oxygen. The coder submits:

- Principal diagnosis: U07.1 (COVID-19)
- Secondary diagnosis: J12.82 (pneumonia due to COVID-19)
- DRG: 177

Coding validation asks, one question at a time:

- Is U07.1 supported by the note as the **principal** (main) reason for admission? (Yes — the record shows COVID-19 drove the admission.)
- Does the official guideline say to sequence the viral infection first? (The guideline *Coding of Sepsis / COVID-19*, section I.C.1.d, says to code the underlying infection first.)
- Are U07.1 and J12.82 allowed together, or does an **Excludes1** note forbid the pair?
- Does DRG 177 correctly follow from this exact set of codes?
- Are the POA flags Y for both diagnoses?

If the documents and the cited guidelines support every answer, the coding validation result is **supported**. If one is contradicted, the result is **not supported**. If something is unclear, the result is **needs human review**.

The two ways coding goes wrong

- **Upcoding** — billing a more severe or expensive code than the record supports. This is a compliance and audit risk: overpayments must be refunded and penalties can follow.
- **Downcoding** — missing a valid complication or comorbidity code, so the claim lands in a lower-paying DRG. This is pure lost revenue.

Good coding validation protects against both.

Where the "rulebook" lives

The system stores the official guideline text in a graph database (**Neo4j**). For every code in the case it retrieves the relevant guideline sections and hands them to the AI, which must **cite the exact guideline text** for each judgment it makes. No citation, no trust.

1.2 What Is Clinical Validation?

Coding validation is about the paperwork. **Clinical validation is about the patient's body**. It sets the wording of the note aside and asks a harder question:

"Do the measurable clinical facts — labs, vital signs, infection status — actually meet the official medical definition of the condition we coded?"

The system's live example is **sepsis**, because "sepsis" is one of the most frequently disputed diagnoses in audits.

The official definition (Sepsis-3)

Sepsis is not simply "the doctor wrote sepsis." The international Sepsis-3 definition requires **both** of these to be true:

1. There is a **suspected or confirmed infection**, and
2. Organ dysfunction has **acutely worsened by 2 or more points** on the **SOFA** score compared with the patient's baseline.

SOFA ("Sequential Organ Failure Assessment") is a single number built from six organ systems. Each system scores 0–4, and the six scores are added together:

Organ system	What is measured
Lungs	Oxygenation (PaO ₂ /FiO ₂ ratio)
Blood clotting	Platelet count
Liver	Bilirubin
Heart / blood pressure	Mean arterial pressure + vasopressor drugs
Brain	Glasgow Coma Scale (level of consciousness)
Kidneys	Creatinine and urine output

The engine reads the patient's labs and vitals, scores each system, sums them, and subtracts the baseline. If the increase (Δ -SOFA) is **2 or more**, the Sepsis-3 definition is met.

Three worked examples

Example A — sepsis is clinically supported → KEEP

- Infection: yes (positive blood culture)
- Platelets 80 → score 2
- Creatinine 2.2 → score 2
- MAP 65 → score 1
- Remaining systems normal → 0
- Total SOFA = 5; baseline = 0; **delta = 5 ≥ 2** → definition met → the sepsis code is supported.

Example B — sepsis is not supported → REMOVE_OR_REVIEW

- Infection: yes
- Platelets 220 , creatinine 1.0 , MAP 85 , GCS 15 , bilirubin normal
- Total SOFA = 0; baseline = 0; **delta = 0 < 2** → definition **not** met → the sepsis code is not supported by the clinical evidence, even if the note uses the word "sepsis."

Example C — the system cannot decide → HUMAN_REVIEW_REQUIRED

- Infection: suspected, but no SOFA-relevant labs or vitals were recorded (or the case lists "known data gaps").
- Without data the score cannot be trusted, so the system refuses to guess and sends the case to a human.

Why this is worth doing

Sepsis codes sit at the top of most audit hit-lists. A code the labs do not support is a recovery risk, even when the physician's note uses the word. Clinical validation catches that gap automatically, before the claim goes out.

I.3 Why We Need Both

	Clinical validation	Coding validation
Question	Does the patient's evidence meet the medical definition?	Are the codes correct, supported, and correctly sequenced per the official rules?
Evidence used	Labs, vitals, infection status, SOFA score	Diagnosis/procedure codes, DRG, POA flags, documentation, guideline text
Example	Sepsis-3: infection and delta-SOFA $\geq$ 2	U07.1 as principal diagnosis; DRG 177 matches the codes
Authority	Clinical definitions (Sepsis-3)	ICD-10-CM Official Guidelines, CMS DRG rules
Fails when	The biology does not meet the definition	A code is unsupported, mis-sequenced, or paired with the wrong DRG

A claim can pass one and fail the other. A patient can genuinely have sepsis (clinical pass) while the coder selects the wrong DRG (coding fail). Or the codes can be perfectly sequenced (coding pass) while the labs do not actually meet Sepsis-3 (clinical fail). This system runs **both** checks on every case.

I.4 Why AI Alone Is Not Enough

The system uses a large language model (Azure OpenAI) to read the clinical note and the retrieved guidelines and to produce a reasoned verdict with citations. That is powerful — but an LLM can occasionally "hallucinate" a plausible-sounding citation or reach a conclusion that is not reproducible.

So the AI is wrapped by a **deterministic engine** — ordinary code that returns the same answer every time for the same input. It does two jobs:

1. **Computes the objective scores itself** (the SOFA score described in Section I.2), independent of the AI.
2. **Applies eight hard compliance rules** and can **override** the AI.

Think of it as an expert reviewer (the AI) whose work must also pass a mandatory safety checklist (the deterministic engine). The checklist can **upgrade** a result from `PASS` to `HUMAN_REVIEW_REQUIRED`, but it will **never** upgrade a failure to a pass.

The eight rules, in plain language:

Rule	In plain language
R1	Required information is missing → review
R2	Upstream extraction flagged the case for coder review → review
R3	Input warnings (missing POA, low completeness) → review
R4	No guideline could be found for a code → warning
R5	The AI returned no citations → review
R6	Any check the AI made came back false → review
R7	Zero guidelines were retrieved → review
R8	The clinical (SOFA) check said "review" or "remove" → review

When any rule forces review, the case is never automatically passed. This is the **human-in-the-loop** design: when the system is not sure, it says so rather than guessing.

1.5 One Case, Walked Through End to End

Here is everything that happens to a single case — the same `CASE-2026-CV-19` used in the examples above. **No code is required to follow along.**

1. **A case arrives.** You send a JSON description of the case: the codes claimed, the DRG, a short documentation excerpt, and — for clinical validation — the relevant labs and vitals.
2. **The case is checked and cleaned (`CaseParser`).** Required fields (`case_id` , `encounter_id` , `principal_dx` , `drg`) must be present; codes are uppercased, stripped, and de-duplicated. Problems become *errors* (which force human review) or *warnings*.
3. **Clinical validation runs (`ClinicalValidationRegistry` → `SofaEngine`).** If the case's codes match an enabled validator — for example the sepsis trigger codes `A41` or `R65.2` — the Sepsis-3 validator activates and computes SOFA from the labs and vitals, returning `KEEP` , `REMOVE_OR_REVIEW` , or `HUMAN_REVIEW_REQUIRED` . Cases without a matching validator skip this step.
4. **Guidelines are retrieved (`GuidelineRetriever` → `Neo4j`).** For every code in the case, the system looks up the relevant sections of the official coding guidelines.
5. **The prompt is assembled (`PromptBuilder`).** The case, the guidelines (as numbered citations), the required output format, and strict rules are combined into a single prompt. If it is too long, the guideline text is trimmed — never the case.
6. **The AI reads and judges (`ClinicalValidator` → `Azure OpenAI`).** The model returns a structured JSON verdict: four boolean findings (`principal_dx_valid` , `sequencing_valid` , `drg_match_valid` , `poa_consistency_valid`), a reasoning summary, and citations. The response is schema-checked; if it is malformed, the case becomes an error requiring review.
7. **The compliance rules run (`ComplianceValidator`).** The eight rules above are applied. Any trigger forces human review.
8. **The result is recorded (`AuditLogger`).** The full result is written to a JSONL file and to PostgreSQL, with a SHA256 hash and timestamp so it cannot be altered unnoticed.
9. **You get an answer.** The response contains your original case plus a `validation_results` block whose `validation_status` is one of: - `PASS` — everything checked out; the codes are supported and consistent. - `FAIL` — a check came back false or a code is unsupported. -

`HUMAN_REVIEW_REQUIRED` — something was ambiguous, missing, or a rule forced review. - `ERROR` — a system problem (for example, unreadable AI output) occurred.

I.6 How the APIs Work, Step by Step

What an API is

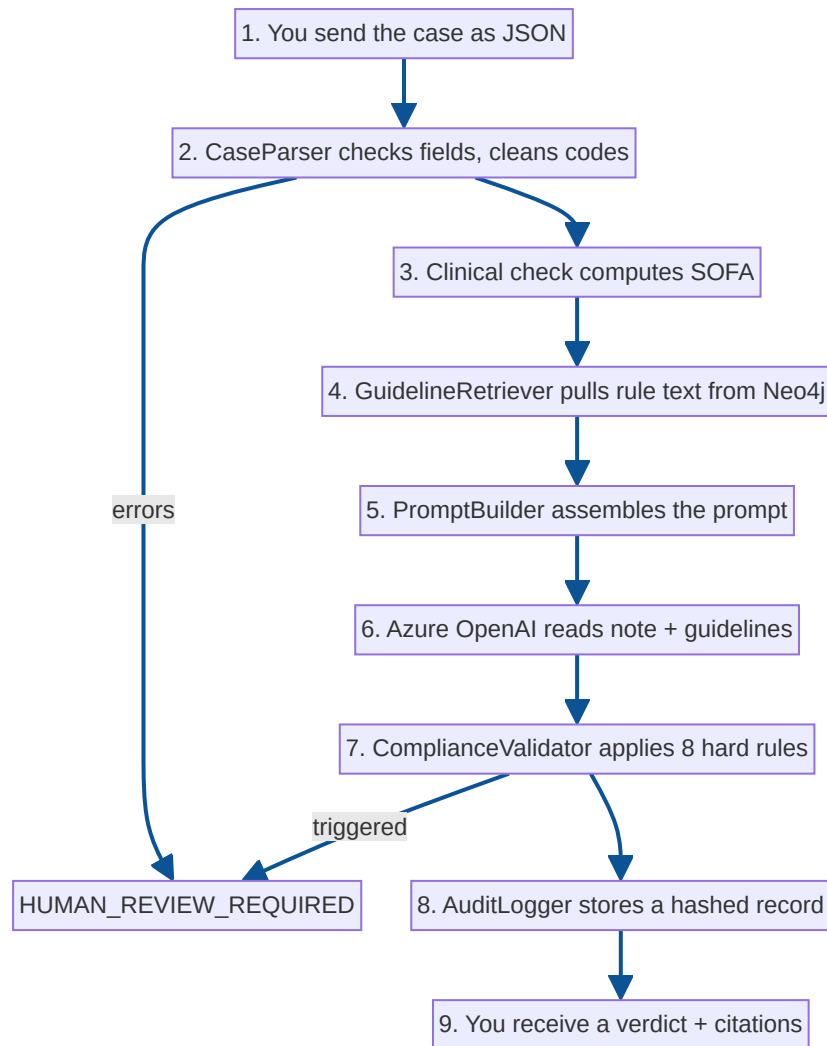
An **API** is a service window: you send a structured request to a specific address (an "endpoint") and get a structured response back. This system exposes its endpoints under the base path `/api/v1`. Every validation endpoint accepts JSON and returns JSON.

The endpoints at a glance

Method & path	What it does	Analogy
POST <code>/api/v1/validate-case</code>	Validate one complete case	"Inspect this one car."
POST <code>/api/v1/convert/nlp-extraction</code>	Translate a raw NLP extraction into the system's case format (no validation)	"Fill in our form from this document."
POST <code>/api/v1/validate-nlp-extraction</code>	Translate and validate in a single call	"Translate and inspect in one step."
POST <code>/api/v1/batch/reprocess-samples</code>	Re-validate many saved cases in parallel	"Re-inspect the whole lot."
GET <code>/api/v1/audit/logs</code>	Search the validation history	"Show me the logbook."
GET <code>/api/v1/audit/cases/{case_id}/logs</code>	Get every past validation for one case	"Show this case's full history."

Sequential walkthrough: validating one case

This is the core flow, step by step. Each step finishes before the next begins, so a failure early on (for example, a missing `case_id`) short-circuits the rest.



1. **Send the request.** `POST /api/v1/validate-case` with the case JSON. An optional `X-Request-ID` header is tracked in the audit log.
2. **Parse and clean.** `CaseParser` verifies the required fields and normalizes codes. Errors here force `HUMAN_REVIEW_REQUIRED` immediately.
3. **Clinical check.** If the codes trigger a validator (sepsis), the SOFA engine scores the labs and vitals now, before the AI is involved.
4. **Retrieve guidelines.** `GuidelineRetriever` queries Neo4j for the guideline sections tied to each code.
5. **Build the prompt.** `PromptBuilder` combines the case, the numbered guideline citations, the required JSON schema, and the strict rules.
6. **Ask the AI.** The model returns a JSON verdict with four findings, a reasoning summary, and citations, which the system validates against the expected schema.
7. **Apply the hard rules.** `ComplianceValidator` runs its eight deterministic checks and can override the AI.
8. **Record the result.** The outcome is hashed and stored in both a JSONL file and PostgreSQL.
9. **Return the answer.** The response repeats the case and adds the `validation_results` block, including the final `validation_status` and the citations used.

The full participant-by-participant interaction is shown as a sequence diagram in [Appendix B.1](#).

The NLP two-step

Sometimes the input is not a tidy case but the raw output of a natural-language processing (NLP) pipeline. That raw output has a different shape (entities, demographics, vitals, vasopressors), so the system offers two options:

- **Two calls (to inspect the middle):** call `POST /api/v1/convert/nlp-extraction` to see the converted case, then call `POST /api/v1/validate-case` with it.
- **One call (fast path):** call `POST /api/v1/validate-nlp-extraction`, which converts and validates together.

The conversion step maps raw entity names onto the expected codes and units but deliberately does **not** invent data: unmapped fields are left out and flagged rather than guessed.

Batch re-processing

`POST /api/v1/batch/reprocess-samples` re-runs validation over many cases at once. It pulls the list of case IDs from PostgreSQL, matches each to its saved sample file, and validates them in a thread pool (`max_workers`, default 8). The response summarises how many cases matched, how many were validated, and each case's result. This is how you re-check a whole book of business after a rule changes.

Part II — Technical Reference

1. Project Overview & Scope

1.1 What is this system?

The Clinical & Coding Validation System is an **AI-powered, guideline-grounded validation engine** for ICD-10-CM/PCS medical coding. It takes a clinical case (diagnosis codes, procedures, DRG claim, documentation), retrieves official coding guidelines from a Neo4j knowledge graph, sends both to an LLM for judgment, and then applies a deterministic compliance layer that can override the AI's output.

The system answers one core question: **"Is the claimed medical code supported by the official coding guidelines and the patient's clinical evidence?"**

1.2 Problem Statement

Clinical coding errors lead to: - **Compliance risk:** Incorrect DRG assignments cause overpayments or underpayments - **Audit exposure:** Unsupported codes fail CMS audits - **Revenue impact:** Missed CC/MCC hierarchies leave money on the table - **Patient safety:** Wrong codes can affect care pathway tracking

Existing tools are either purely rule-based (miss nuance) or purely AI-based (hallucinate citations). This system combines both: an LLM for nuanced guideline interpretation, backed by a deterministic compliance engine that enforces hard rules.

1.3 Scope

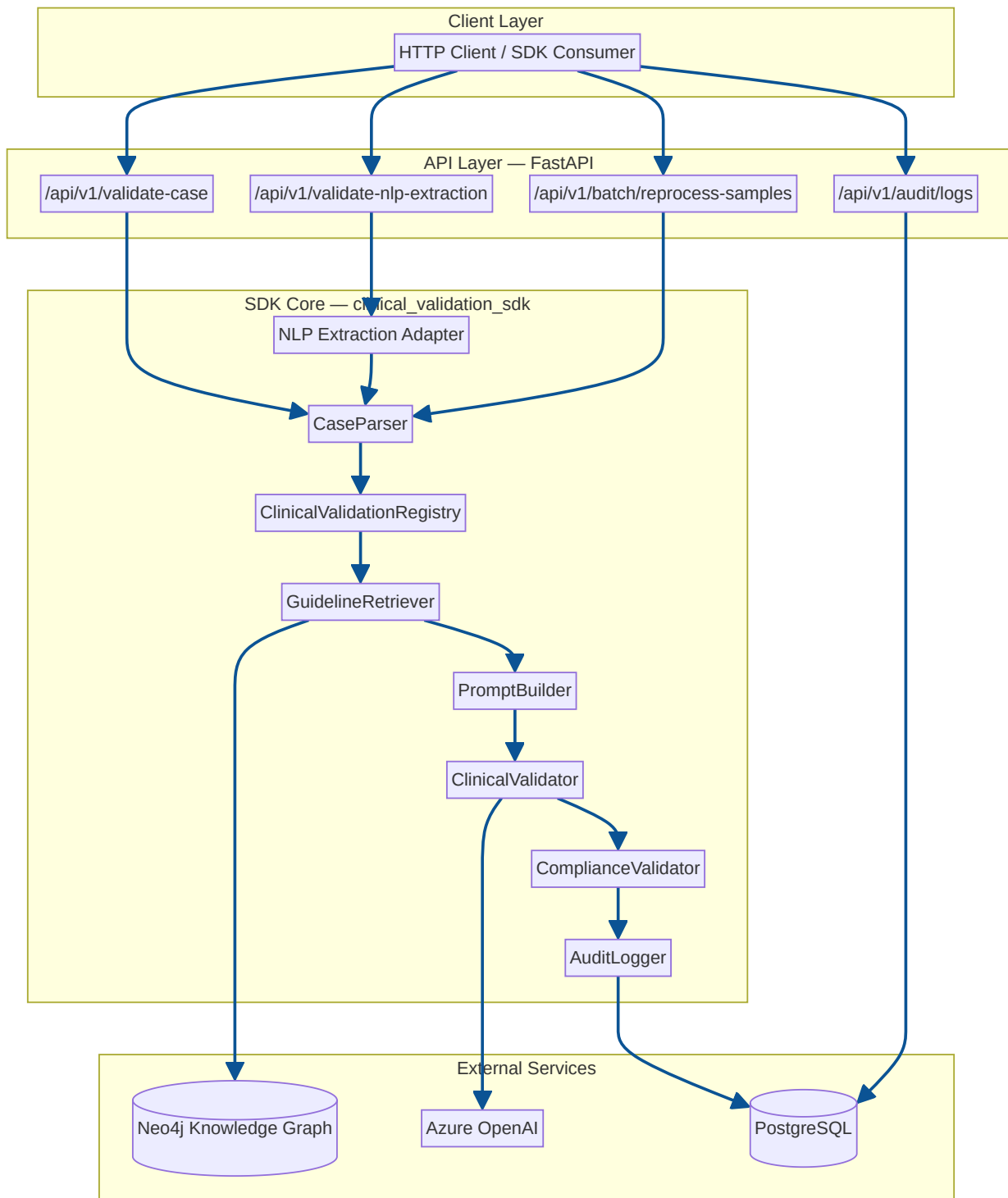
In Scope	Out of Scope
ICD-10-CM diagnosis code validation	CPT/HCPCS procedure code validation (planned)
DRG claim verification	Real-time claim adjudication
Sepsis-3 (SOFA) clinical validation	Full clinical decision support
Neo4j knowledge graph guideline retrieval	Direct EHR integration
Azure OpenAI LLM inference	On-premise LLM deployment
PostgreSQL audit logging	HIPAA-compliant data storage (planned)
Batch reprocessing	Streaming/real-time validation
NLP extraction format adaptation	NLP entity extraction itself

1.4 Key Design Principles

- Guideline-grounded:** Every AI determination must cite specific guideline text — no unsourced judgments
- Deterministic override:** The compliance layer can override the AI output based on hard rules (missing citations, failed findings, insufficient data)
- Human-in-the-loop:** Any ambiguity forces `HUMAN_REVIEW_REQUIRED` rather than guessing
- Clinical extensibility:** Diagnosis-specific validators (sepsis, respiratory failure, malnutrition) are config-driven, not code-driven
- Audit everything:** Every validation is SHA256-hashed, timestamped, and dual-persisted (JSONL + PostgreSQL)

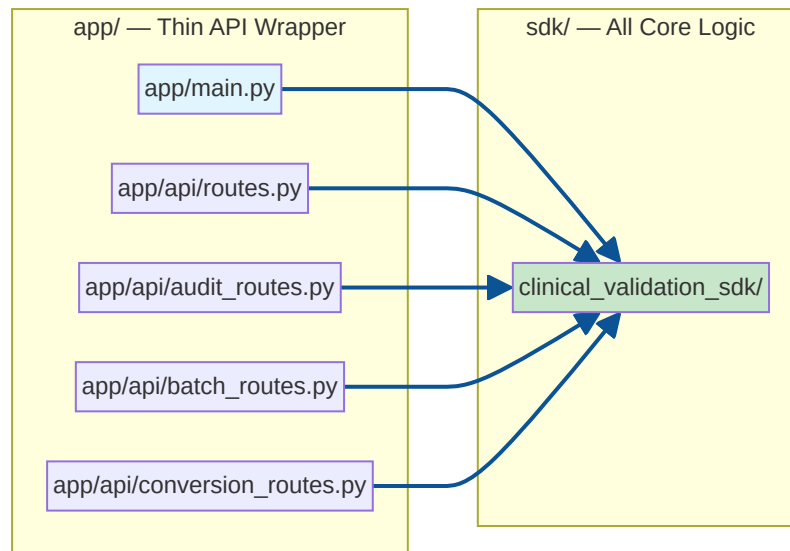
2. Solution Architecture

2.1 High-Level Architecture



2.2 Two-Layer Architecture

The system is deliberately split into two layers:

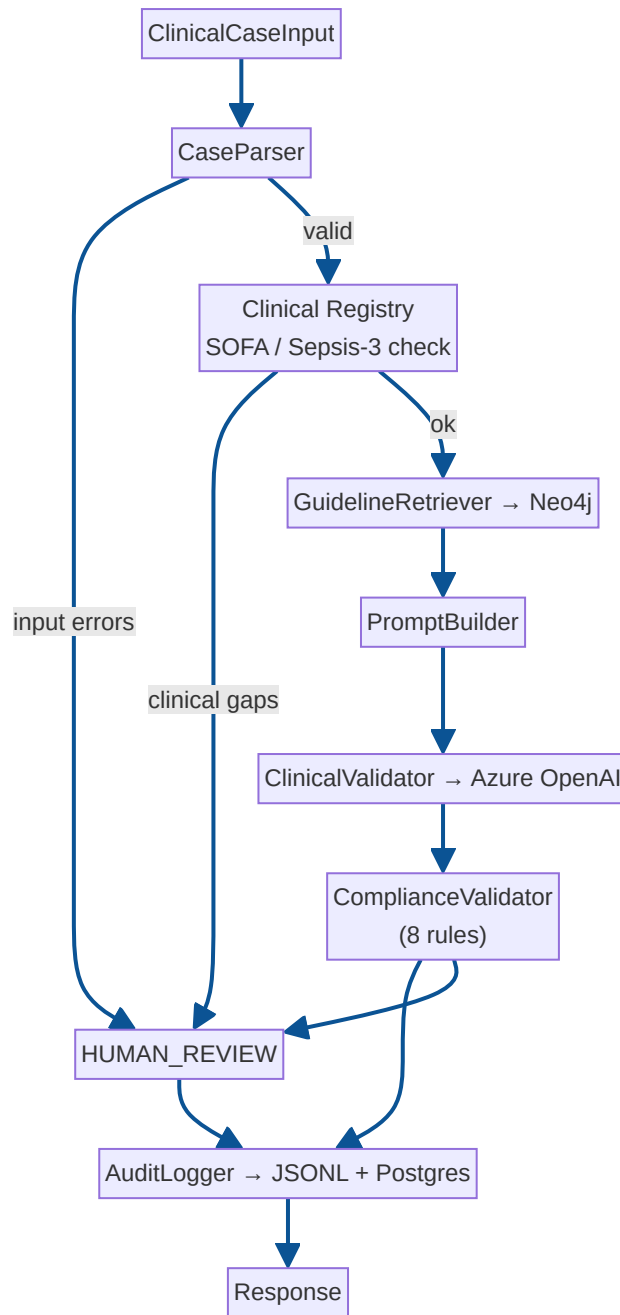


app/ contains only FastAPI route definitions. Every module under **app/** (schemas, services, AI, retrieval, etc.) is a **backward-compatible shim** that re-exports from the SDK:

```
# app/config/settings.py – example shim
from clinical_validation_sdk.config.settings import * # noqa: F401,F403
```

sdk/clinical_validation_sdk/ contains ALL business logic: schemas, parsing, retrieval, AI, compliance, clinical validation, audit, database, and utilities.

2.3 Request Processing Pipeline

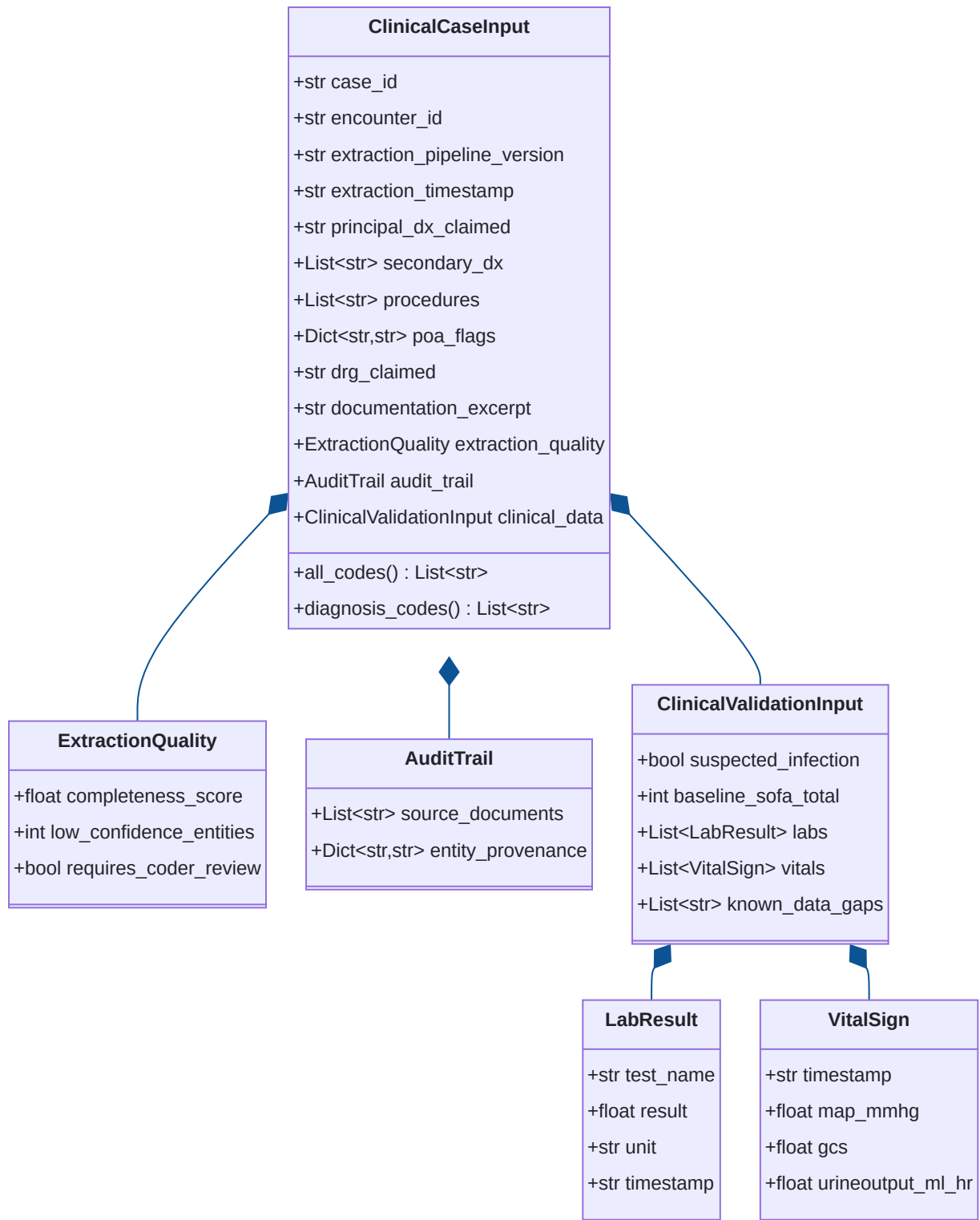


3. Data Model & Case Schema

3.1 Core Entity: `ClinicalCaseInput`

File: `sdk/clinical_validation_sdk/schemas/case_schema.py`

The central data structure that flows through the entire pipeline:

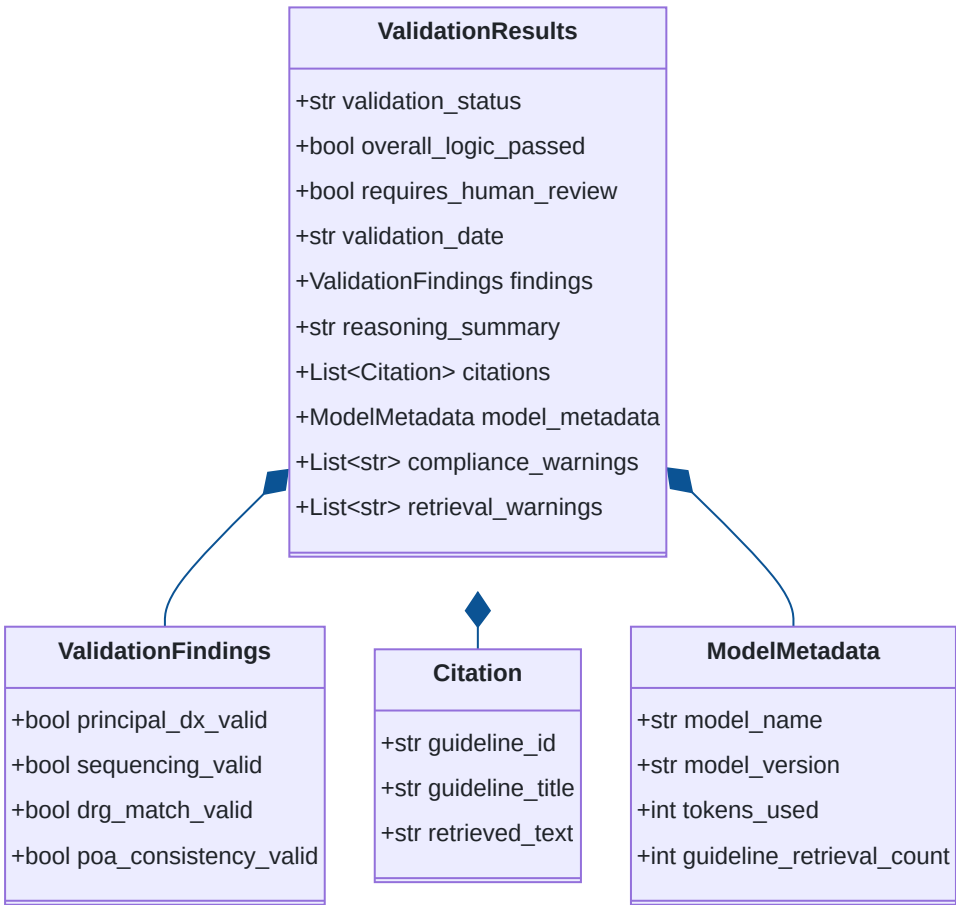


3.2 Field Descriptions

Field	Type	Description
case_id	str	Unique case identifier (e.g., CASE-2026-CV-19)
encounter_id	str	Hospital encounter identifier
principal_dx_claimed	str	Primary ICD-10-CM diagnosis code being validated
secondary_dx	List[str]	Additional diagnosis codes
procedures	List[str]	ICD-10-PCS procedure codes
poa_flags	Dict[str,str]	Present-on-Admission flags per code (Y/N/U)
drg_claimed	str	Claimed DRG code
documentation_excerpt	str	Clinical note text (max 20,000 chars)
clinical_data	ClinicalValidationInput	Labs, vitals, infection status for SOFA scoring
extraction_quality	ExtractionQuality	Upstream extraction confidence metrics

3.3 Validation Result Schema

File: sdk/clinical_validation_sdk/schemas/validation_schema.py



Possible validation_status values: - PASS — All findings valid, guidelines support codes, no compliance overrides - FAIL — One or more findings invalid or unsupported -

HUMAN_REVIEW_REQUIRED — Ambiguity, missing data, or compliance rule triggered review - ERROR —
System error during validation

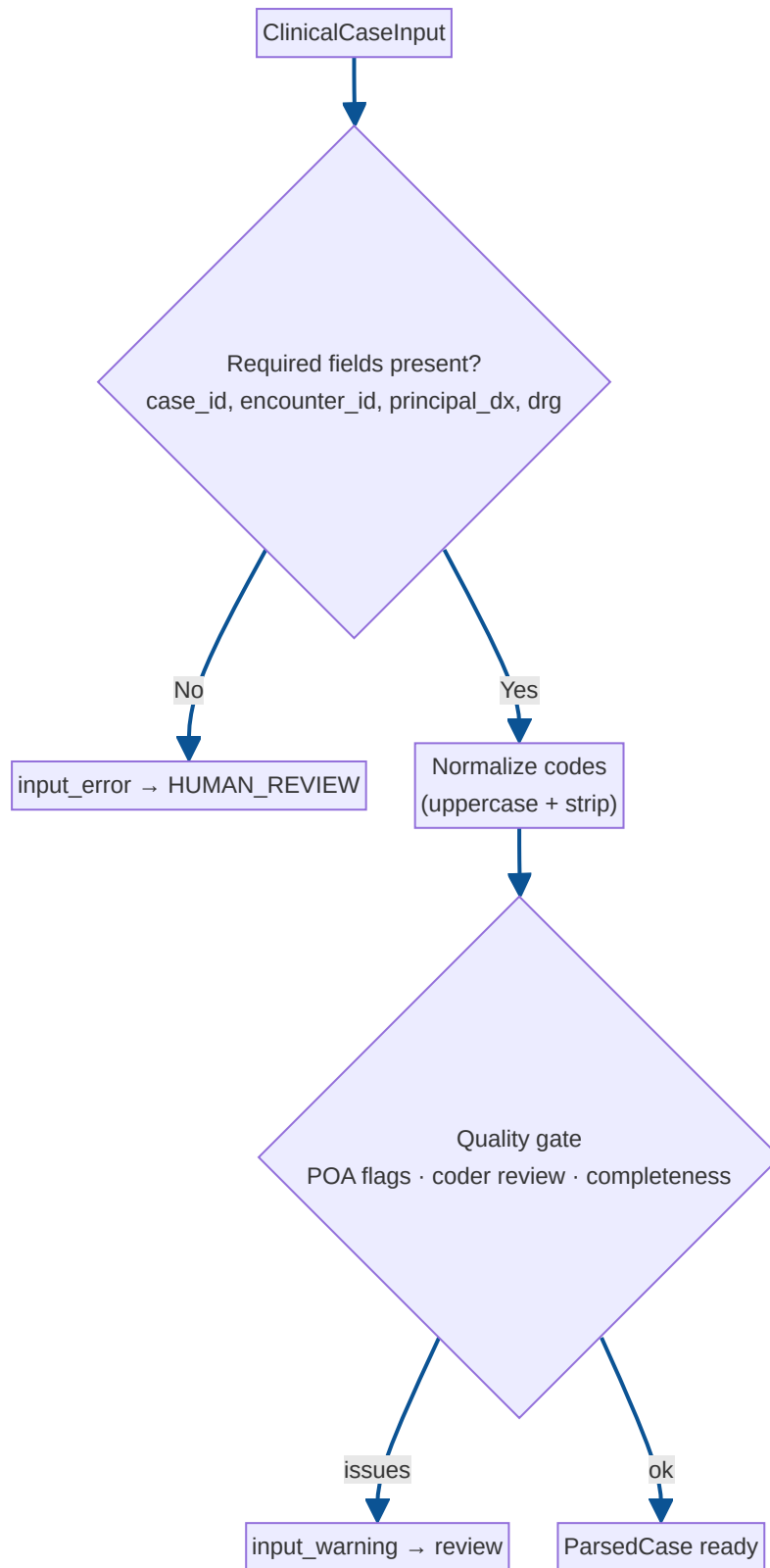
4. Ingestion & Case Parsing Layer

File: `sdk/clinical_validation_sdk/ingestion/case_parser.py`

4.1 Purpose

The `CaseParser` normalizes and validates incoming `ClinicalCaseInput` before it enters the pipeline. It produces a `ParsedCase` containing the normalized case, all extracted codes, and any input errors or warnings.

4.2 Parsing Logic



4.3 Error vs. Warning Handling

Condition	Severity	Effect
Missing <code>case_id</code>	Error	Forces <code>HUMAN_REVIEW_REQUIRED</code>
Missing <code>encounter_id</code>	Error	Forces <code>HUMAN_REVIEW_REQUIRED</code>
Missing <code>principal_dx_claimed</code>	Error	Forces <code>HUMAN_REVIEW_REQUIRED</code>
Missing <code>drg_claimed</code>	Error	Forces <code>HUMAN_REVIEW_REQUIRED</code>
Missing POA flag for dx code	Warning	Forces review if <code>FORCE_REVIEW_ON_INPUT_WARNINGS</code>
Empty <code>documentation_excerpt</code>	Warning	Forces review if <code>FORCE_REVIEW_ON_INPUT_WARNINGS</code>
<code>requires_coder_review = true</code>	Warning	Forces human review
<code>completeness_score < 0.75</code>	Warning	Forces review if <code>FORCE_REVIEW_ON_INPUT_WARNINGS</code>

4.4 Code Normalization

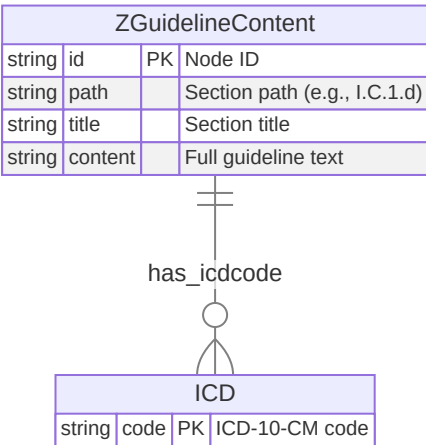
All diagnosis and procedure codes are: - **Uppercased:** `"u07.1"` → `"U07.1"` - **Stripped:** `" U07.1 "` → `"U07.1"` - **Deduplicated:** while preserving order via `dict.fromkeys()`

The `all_codes` property combines `principal_dx_claimed` + `secondary_dx` + `procedures` into a single list for guideline retrieval.

5. Knowledge Graph & Guideline Retrieval

5.1 Neo4j Schema

The knowledge graph uses two node types connected by a relationship:



Example Cypher data:

```
CREATE (i:ICD {code: "A41.9"})
CREATE (z:ZGuidelineContent {
  id: " guideline_z_3",
  path: "I.C.1.d",
  title: "Coding of Sepsis, Severe Sepsis and Septic Shock",
  content: "Code first the underlying systemic infection..."
})
CREATE (z)-[:has_icdcode]->(i)
```

5.2 Retrieval Query

File: sdk/clinical_validation_sdk/retrieval/guideline_retriever.py

```
UNWIND $codes AS submitted_code
WITH submitted_code, left(submitted_code, 3) AS category
MATCH (i:ICD)
WHERE i.code = submitted_code OR i.code = category
MATCH (z:ZGuidelineContent)-[:has_icdcode]->(i)
RETURN DISTINCT
  z.id AS node_id,
  z.path AS guideline_id,
  z.title AS guideline_title,
  z.content AS retrieved_text,
  submitted_code AS matched_code
```

Key behaviors: - **Category matching:** If exact code isn't found, falls back to the 3-character prefix (e.g., A41.9 tries A41 as well) - **Per-code cap:** Max MAX_GUIDELINES_PER_CODE (default: 3) sections per code - **Text truncation:** Guideline text capped at MAX_GUIDELINE_CHARS (default: 2000) with ...[truncated] appended - **Warning generation:** One warning per code with no guidelines, plus one if results were capped

5.3 Neo4j Client

File: sdk/clinical_validation_sdk/retrieval/neo4j_client.py

Feature	Detail
Driver	neo4j.GraphDatabase with basic_auth
Retry	tenacity : 3 attempts, exponential backoff (max 8s)
Timeouts	Connection: 10s, Transaction retry: 15s
Database	Configurable via NE04J_DATABASE (default: neo4j)
Health check	RETURN 1 AS ok query
Connectivity	Verified at init, logged but not fatal on failure

6. AI Integration & Prompt Engineering

6.1 Prompt Template

File: `sdk/clinical_validation_sdk/prompts/templates/clinical_validation_prompt.txt`

The prompt follows a strict structure:

System: "You are a strict clinical coding compliance validator. Return only valid JSON."

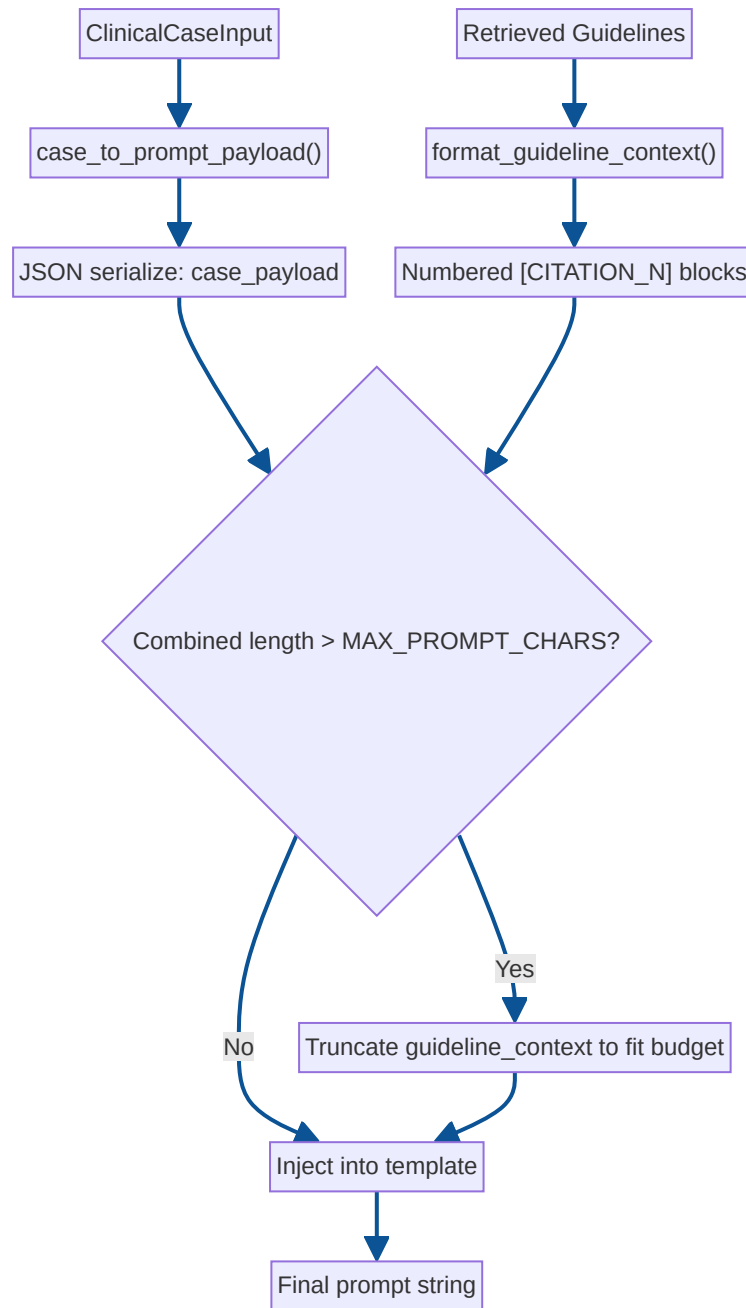
User prompt:

1. Role definition + 10 strict rules
2. Case payload (JSON)
3. Retrieved guideline context (numbered citations)
4. Required JSON output format (schema)
5. Example JSON output (exact format to follow)

Critical rules enforced in the prompt: - Every determination MUST include at least one guideline citation - Do NOT generate alternate DRGs - Do NOT recommend coding changes - Do NOT suggest documentation improvements - If guideline support is missing, set `requires_human_review=true` - Return ONLY valid JSON, no markdown

6.2 Prompt Construction

File: `sdk/clinical_validation_sdk/prompts/prompt_builder.py`



Budget management: If case JSON + guideline context exceeds `MAX_PROMPT_CHARS` (default: 30,000), the guideline context is truncated to fit, preserving the full case payload.

6.3 Guideline Context Formatting

Each retrieved guideline is formatted as:

```
[CITATION_1]
guideline_id: I.C.1.d
guideline_title: Coding of Sepsis
matched_code: A41.9
page_start: None
retrieved_text:
Code first the underlying systemic infection...
```

6.4 Azure OpenAI Client

File: sdk/clinical_validation_sdk/ai/azure_openai_client.py

Setting	Value
Model	Configurable via <code>AZURE_OPENAI_DEPLOYMENT_NAME</code>
Temperature	0 (deterministic)
Response format	<code>{"type": "json_object"}</code>
Retry	tenacity : 3 attempts, exponential backoff (max 10s)
API version	2024-12-01-preview (default)

Output parsing: The client uses `safe_json_loads()` which handles markdown code fences (````json ... ````) that some models wrap around their JSON output.

6.5 Model Output Validation

File: sdk/clinical_validation_sdk/ai/clinical_validator.py

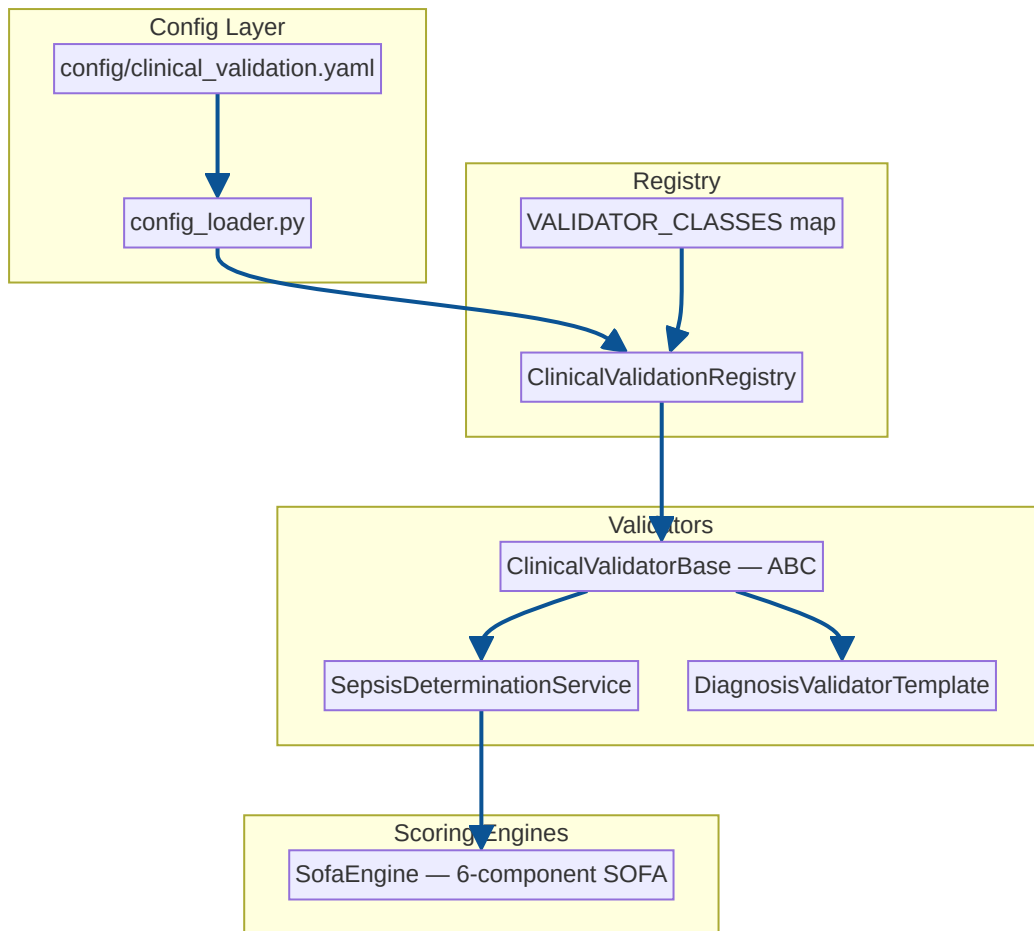
After receiving the AI response, the `ClinicalValidator` validates the output structure:

Required Key	Type Check
<code>validation_status</code>	<code>str</code> in
<code>overall_logic_passed</code>	<code>bool</code>
<code>requires_human_review</code>	<code>bool</code>
<code>findings.principal_dx_valid</code>	<code>bool</code>
<code>findings.sequencing_valid</code>	<code>bool</code>
<code>findings.drg_match_valid</code>	<code>bool</code>
<code>findings.poa_consistency_valid</code>	<code>bool</code>
<code>citations</code>	<code>list</code>

If any check fails, the output is replaced with an `ERROR` status requiring human review.

7. Deterministic Clinical Validation Engine

7.1 Architecture



7.2 Gated Validator Pattern

File: `sdk/clinical_validation_sdk/clinical/registry.py`

Validators are **gated** — enabled/disabled entirely by configuration:

```
# config/clinical_validation.yaml
clinical_validators:
  sepsis:
    enabled: true
    trigger_codes: ["A41", "R65.2"]
    params:
      delta_threshold: 2
  acute_respiratory_failure:
    enabled: false
    trigger_codes: ["J96"]
  malnutrition:
    enabled: false
    trigger_codes: ["E43", "E44"]
```

Adding a new validator requires ZERO code changes to the pipeline:

1. Implement `evaluate()` in a new subclass of `ClinicalValidatorBase`
2. Add the class to `VALIDATOR_CLASSES` in `registry.py`
3. Add/enable its block in `clinical_validation.yaml`

7.3 Sepsis-3 (SOFA) Clinical Validation

Files: - `sdk/clinical_validation_sdk/clinical/sepsis_service.py` — Decision layer -
`sdk/clinical_validation_sdk/clinical/sofa_engine.py` — Scoring engine

Sepsis-3 Criteria

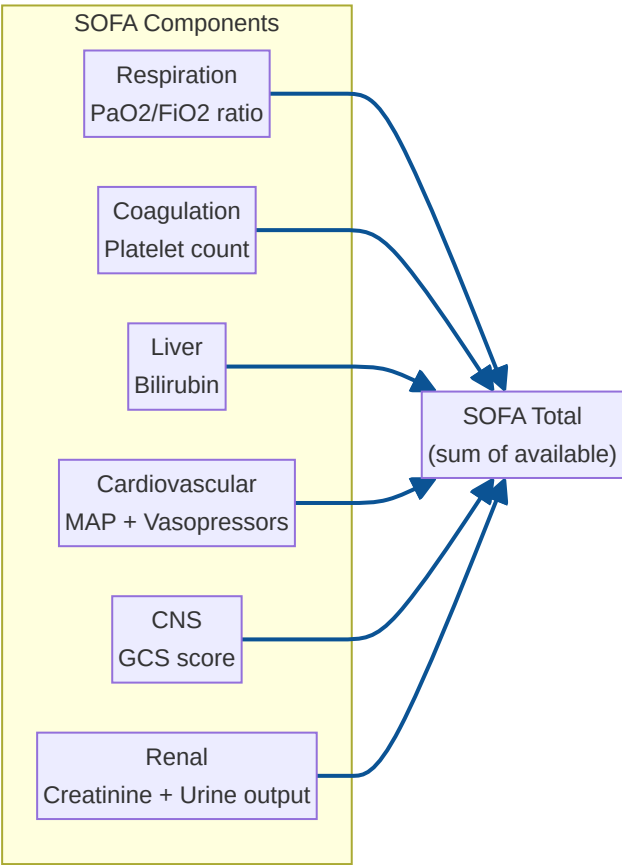
Sepsis = Suspected/confirmed infection + Acute SOFA increase $\geq$ 2 points from baseline

Clinical Decision Matrix

Condition	Clinical Decision	Validation Status
Infection present + delta-SOFA $\geq$ 2	KEEP	Code supported
No documented infection	REMOVE_OR_REVIEW	Code not supported
Infection present but delta-SOFA $<$ 2	REMOVE_OR_REVIEW	Code not supported
Missing labs/vitals entirely	HUMAN_REVIEW_REQUIRED	Cannot evaluate
Labs present but none match SOFA components	HUMAN_REVIEW_REQUIRED	Cannot evaluate
known_data_gaps non-empty	HUMAN_REVIEW_REQUIRED	SOFA may be understated

7.4 SOFA Engine — 6-Component Scoring

File: `sdk/clinical_validation_sdk/clinical/sofa_engine.py`



Scoring Thresholds

Respiration (PaO₂/FiO₂)

Score	Threshold
0	> 400
1	<= 400
2	<= 300
3	<= 200 + respiratory support
4	<= 100 + respiratory support

Coagulation (Platelets x10³/uL)

Score	Threshold
0	>= 150
1	< 150
2	< 100
3	< 50
4	< 20

Liver (Bilirubin mg/dL)

Score	Threshold
0	< 1.2
1	1.2 - 1.9
2	2.0 - 5.9
3	6.0 - 11.9
4	>= 12.0

Cardiovascular (MAP + Vasopressors)

Score	Condition
0	MAP >= 70 mmHg, no vasopressors
1	MAP < 70 mmHg
2	Dobutamine any dose OR Dopamine <= 5 mcg/kg/min
3	Dopamine > 5 OR Norepi/Epi <= 0.1 mcg/kg/min
4	Dopamine > 15 OR Norepi/Epi > 0.1 mcg/kg/min

CNS (Glasgow Coma Scale)

Score	GCS
0	15
1	13 - 14
2	10 - 12
3	6 - 9
4	< 6

Renal (Creatinine + Urine Output)

Score	Condition
0	Creatinine < 1.2 mg/dL
1	1.2 - 1.9
2	2.0 - 3.4
3	3.5 - 4.9 OR Urine output < 500 mL/day
4	>= 5.0 OR Urine output < 200 mL/day

Total SOFA Computation

- Available component scores are **summed** (None/missing components are excluded)
- `delta_sofa = sofa_total - baseline_sofa_total` (baseline defaults to 0)
- If `known_data_gaps` is non-empty, result is forced to `HUMAN_REVIEW_REQUIRED` regardless of score (SOFA may be understated)

7.5 Test Name Matching

The SOFA engine uses fuzzy matching for lab/vital names via `SOFA_VARIABLE_CONFIG` :

```
SOFA_VARIABLE_CONFIG = {  
    "pao2_fio2": ["pao2/fio2", "pao2/fio2 ratio", "p/f ratio", ...],  
    "platelets": ["platelets", "platelet count", "plt", ...],  
    "bilirubin": ["bilirubin", "total bilirubin", "tbili", ...],  
    "creatinine": ["creatinine", "serum creatinine", "scr", ...],  
    # ... etc  
}
```

Each component takes the **most recent** matching lab/vital by timestamp.

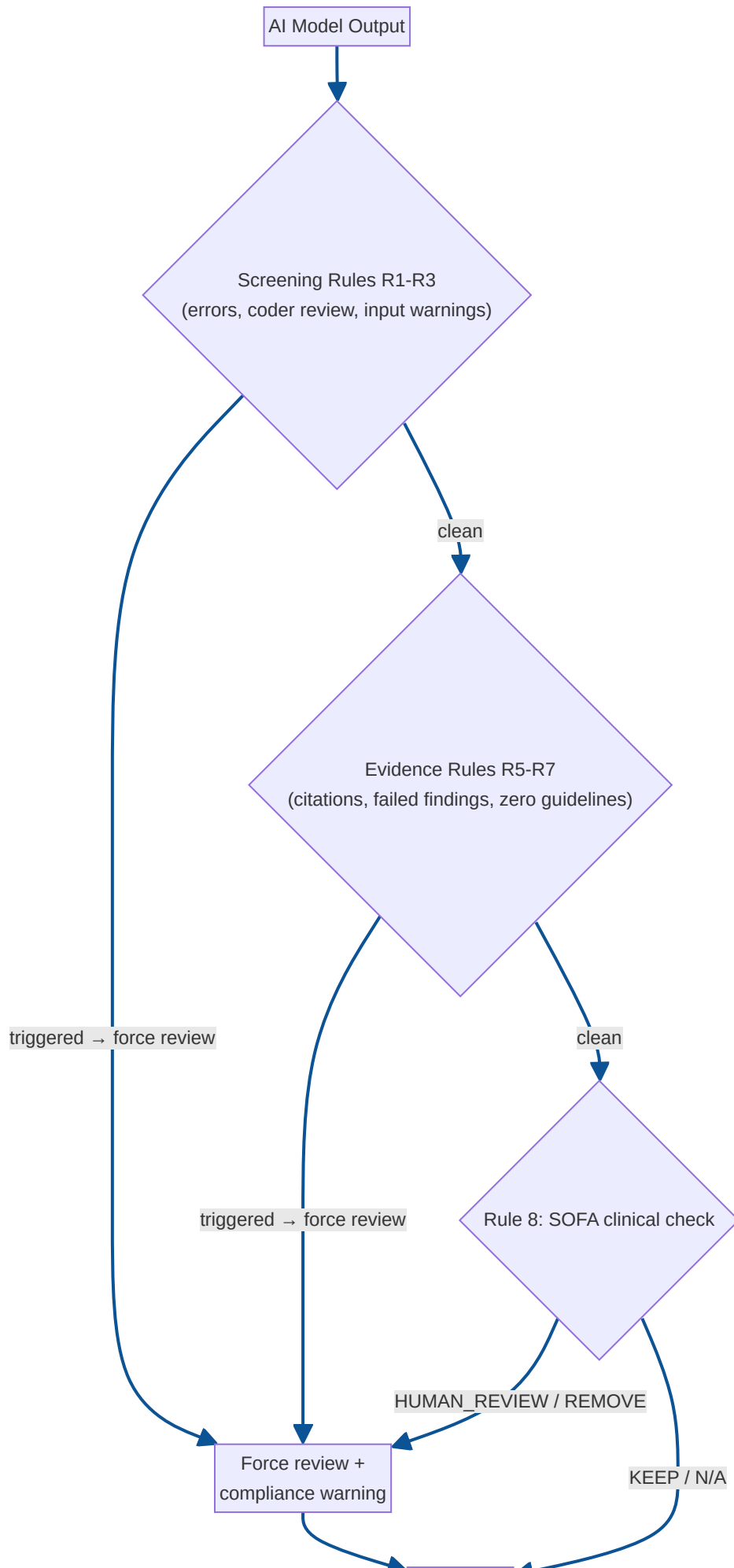
8. Compliance Rules Engine

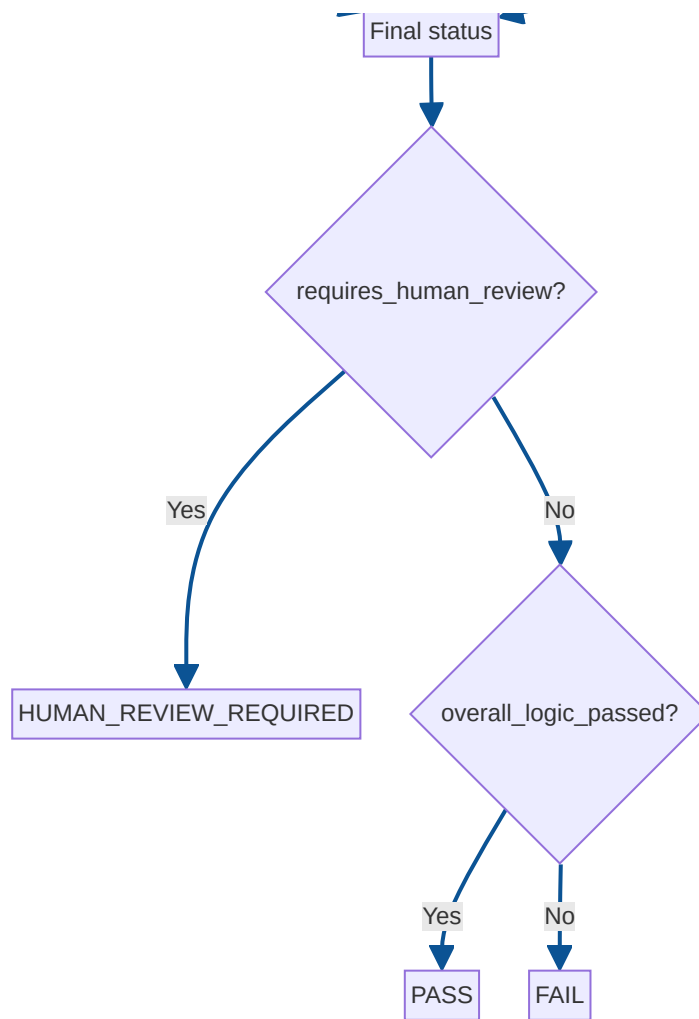
File: `sdk/clinical_validation_sdk/compliance/compliance_validator.py`

8.1 Purpose

The compliance validator is the **final gatekeeper** that can override the AI's output. It applies 8 deterministic rules that force `HUMAN_REVIEW_REQUIRED` or `FAIL` regardless of what the model said.

8.2 The 8 Rules





8.3 Rule Summary Table

Rule	Trigger	Effect
R1	Input errors (missing required fields)	<code>requires_human_review = True</code> , <code>overall_logic_passed = False</code>
R2	<code>extraction_quality.requires_coder_review = true</code>	<code>requires_human_review = True</code>
R3	Input warnings (missing POA, low completeness)	Adds warnings; forces review if <code>FORCE_REVIEW_ON_INPUT_WARNINGS</code>
R4	Neo4j retrieval warnings (no guidelines for code)	Adds warnings (informational)
R5	No citations in AI output	Adds warning; forces review if <code>FORCE_REVIEW_ON_NO_CITATIONS</code>
R6	Any findings boolean is False	Adds warning; forces review if <code>FORCE_REVIEW_ON_FAILED_FINDINGS</code>
R7	<code>guideline_retrieval_count = 0</code>	Adds warning; forces review if <code>FORCE_REVIEW_ON_ZERO_GUIDELINES</code>
R8	Clinical SOFA check result is <code>HUMAN_REVIEW_REQUIRED</code> or <code>REMOVE_OR_REVIEW</code>	Adds warning; forces review

8.4 Final Status Determination

After all rules are applied:

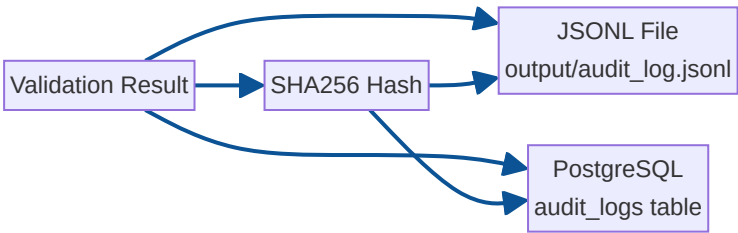
```
if requires_human_review:
    status = "HUMAN_REVIEW_REQUIRED"
elif overall_logic_passed:
    status = "PASS"
else:
    status = "FAIL"
```

This means the compliance layer can **upgrade** (AI said PASS, but rules force HUMAN_REVIEW) or **confirm** (AI said PASS, rules agree) the AI's output, but never **downgrade** to PASS.

9. Audit Logging & Tamper Evidence

File: sdk/clinical_validation_sdk/audit/audit_logger.py

9.1 Dual Persistence Strategy



Storage	Purpose	Format
JSONL file	Quick inspection, backup, backward compatibility	One JSON object per line
PostgreSQL	Production audit trail, queryable, indexed	audit_logs table with UUID PK

9.2 Audit Record Fields

Field	Type	Description
id	UUID	Primary key
case_id	String(255)	Clinical case identifier
validation_status	String(50)	PASS/FAIL/HUMAN_REVIEW_REQUIRED/ERROR
sha256_hash	String(64)	SHA256 of full output JSON (tamper evidence)
output_path	Text	Path to validation result file
guideline_retrieval_count	Integer	Number of guidelines retrieved
created_at	DateTime(UTC)	Record creation timestamp
error_message	Text	Error details if validation failed
error_type	String(255)	Error classification
request_id	String(255)	Request tracking ID
latency_ms	Integer	End-to-end latency
model_inference_ms	Integer	LLM inference time
model_name	String(255)	Model identifier
model_version	String(50)	API version
tokens_used	Integer	Token consumption
clinical_applicable	Boolean	Whether SOFA check applied
sepsis_trigger_codes	JSONB	Trigger codes that fired
sofa_total	Integer	Computed SOFA score
delta_sofa	Integer	SOFA - baseline
sepsis_criteria_met	Boolean	Sepsis-3 criteria satisfied
clinical_decision	String(50)	KEEP/REMOVE_OR_REVIEW/HUMAN_REVIEW_REQUIRED
clinical_validation_details	JSONB	Full clinical validation payload

9.3 Database Schema

```
CREATE TABLE audit_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  case_id VARCHAR(255) NOT NULL,  
  validation_status VARCHAR(50) NOT NULL,  
  output_path TEXT,  
  sha256_hash VARCHAR(64) NOT NULL,  
  guideline_retrieval_count INTEGER NOT NULL DEFAULT 0,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  error_message TEXT,  
  error_type VARCHAR(255),  
  request_id VARCHAR(255),  
  latency_ms INTEGER,  
  model_inference_ms INTEGER,  
  model_name VARCHAR(255),  
  model_version VARCHAR(50),  
  tokens_used INTEGER DEFAULT 0,  
  clinical_applicable BOOLEAN,  
  sepsis_trigger_codes JSONB,  
  sofa_total INTEGER,  
  delta_sofa INTEGER,  
  sepsis_criteria_met BOOLEAN,  
  clinical_decision VARCHAR(50),  
  clinical_validation_details JSONB  
);  
  
-- Composite indexes  
CREATE INDEX idx_case_status_created ON audit_logs (case_id, validation_status, created_at);  
CREATE INDEX idx_status_created ON audit_logs (validation_status, created_at);  
CREATE INDEX idx_created_at ON audit_logs (created_at);  
CREATE INDEX idx_request_id ON audit_logs (request_id);  
CREATE INDEX idx_clinical_decision ON audit_logs (clinical_decision);
```

9.4 Optional Audit Redaction

When `ENABLE_AUDIT_REDACTION=true`, specified keys in the output are replaced with `[REDACTED]` before persistence:

```
AUDIT_REDACT_KEYS = "documentation_excerpt,patient_name,patient_number,provider,medicalRecord"
```

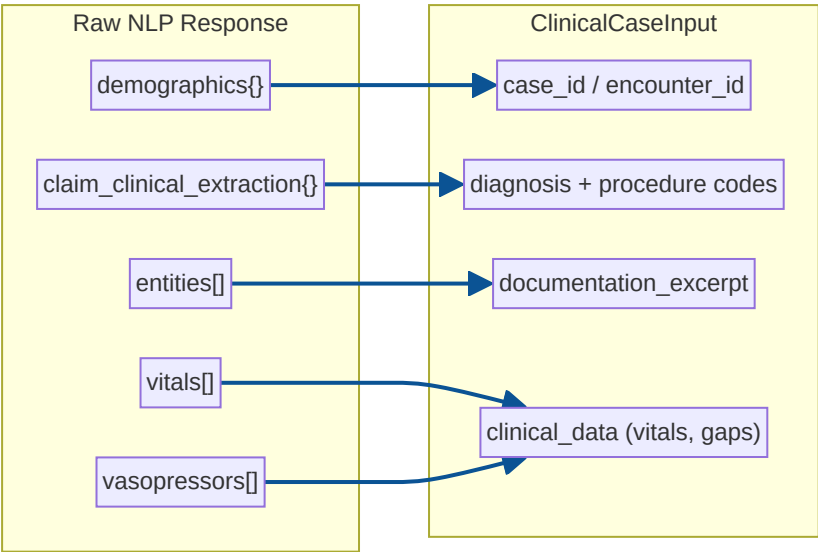
10. NLP Extraction Adapter

File: `sdk/clinical_validation_sdk/adapters/nlp_extraction_adapter.py`

10.1 Purpose

Bridges the gap between a raw NLP clinical-entity-extraction pipeline response and the `ClinicalCaseInput` shape the validation system expects. This is a **format adapter**, not an NLP system.

10.2 Input Format → Output Format Mapping



10.3 Key Adapter Behaviors

Behavior	Detail
Code selection	First candidate with non-null code wins (<code>select_first_coded_candidate</code>)
POA flags	Left empty (source format has none) → triggers review
DRG claimed	Left empty (source format has none) → triggers input error
Documentation	Synthesized from entity list, clearly labeled as non-literal
Urine output	Converted from daily (mL/24h) to hourly (mL/hr) for SOFA engine
Vasopressor gaps	Active vasopressors with no numeric dose → <code>known_data_gaps</code>
Sepsis infection	Defaults to <code>False</code> with a <code>known_data_gap</code> if sepsis code present
Completeness	<code>matched_entities / total_entities</code>

10.4 Deliberate Scope Limits

These are documented decisions, not bugs:

1. **No POA flags** — source format doesn't have them → forces review
2. **No DRG claim** — source format doesn't have them → triggers input error
3. **No raw narrative text** — `documentation_excerpt` is synthesized from entities
4. **No reliable `suspected_infection`** — defaults to `False` , forces review for sepsis codes
5. **No respiration SOFA** — PaO2/FiO2 cannot be reliably paired from scattered entities

11. SDK Extraction & Reusable Package

11.1 Package Structure

The SDK is a standalone Python package (`clinical-validation-sdk`) installable via:

```
pip install -e ./sdk
```

sdk/pyproject.toml :

```
[project]
name = "clinical-validation-sdk"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "pydantic>=2.0",
    "python-dotenv",
    "neo4j>=5.20.0",
    "openai",
    "tenacity",
    "loguru",
    "PyYAML>=6.0",
    "sqlalchemy>=2.0.0",
    "psycopg2-binary>=2.9.0",
]
```

11.2 Public API

File: `sdk/clinical_validation_sdk/client.py`

```
from clinical_validation_sdk import ClinicalValidationClient

client = ClinicalValidationClient()

# Validate a single case
result = client.validate_case(case_dict_or_model)

# Convert raw NLP extraction to ClinicalCaseInput
case = client.convert_nlp_extraction(raw_nlp_response)

# Convert + validate in one call
result = client.validate_nlp_extraction(raw_nlp_response)

# Batch reprocess from Postgres audit logs
batch = client.reprocess_batch(case_ids=["CASE-1", "CASE-2"])

# Query audit trail
logs = client.query_audit_logs(filters=..., skip=0, limit=10)
```

11.3 SDK Module Map

Module	Purpose
client.py	Public facade
schemas/	Pydantic models (case, validation, batch)
config/settings.py	Environment-driven settings
ingestion/case_parser.py	Input normalization
retrieval/neo4j_client.py	Neo4j driver
retrieval/guideline_retriever.py	Cypher queries
prompts/prompt_builder.py	Template-based prompts
ai/azure_openai_client.py	Azure OpenAI wrapper
ai/clinical_validator.py	Model output validation
clinical/base.py	Validator ABC
clinical/registry.py	Gated dispatch
clinical/sepsis_service.py	Sepsis-3 logic
clinical/sofa_engine.py	SOFA scoring
clinical/config_loader.py	YAML config loader
compliance/compliance_validator.py	8-rule compliance
audit/audit_logger.py	Dual persistence
adapters/nlp_extraction_adapter.py	Format adaptation
database/	SQLAlchemy models, repo, service
utils/	JSON, logging helpers

11.4 Backward Compatibility

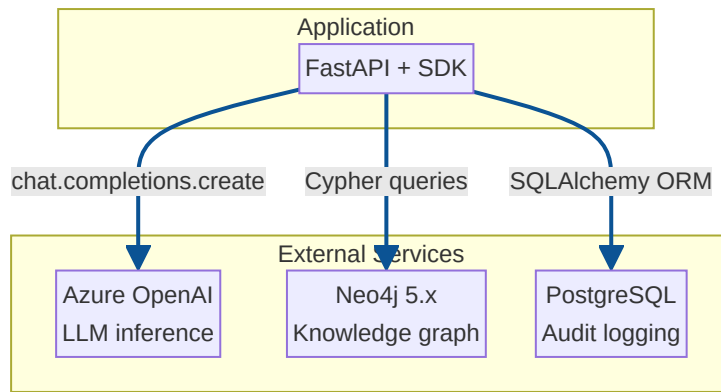
All modules under `app/` are shims:

```
# app/ai/clinical_validator.py
from clinical_validation_sdk.ai.clinical_validator import * # noqa: F401,F403
```

This means existing imports like `from app.services.validation_service import ValidationService` continue to work while the real code lives in the SDK.

12. Infrastructure & DevOps

12.1 External Service Dependencies



12.2 Environment Variables

Variable	Default	Description
AZURE_OPENAI_ENDPOINT	—	Azure OpenAI endpoint URL
AZURE_OPENAI_API_KEY	—	API key
AZURE_OPENAI_DEPLOYMENT_NAME	—	Model deployment name
AZURE_OPENAI_API_VERSION	2024-12-01-preview	API version
NEO4J_URI	bolt://localhost:7687	Neo4j bolt URI
NEO4J_USERNAME	—	Neo4j username
NEO4J_PASSWORD	—	Neo4j password
NEO4J_DATABASE	neo4j	Neo4j database name
POSTGRES_HOST	localhost	PostgreSQL host
POSTGRES_PORT	5432	PostgreSQL port
POSTGRES_DB	postgres	Database name
POSTGRES_USER	postgres	Database user
POSTGRES_PASSWORD	admin	Database password
OUTPUT_DIR	output	Validation result files
MIN_COMPLETENESS_SCORE	0.75	Minimum extraction quality
MAX_GUIDELINE_CHARS	2000	Guideline text truncation limit
MAX_GUIDELINES_PER_CODE	3	Max guidelines per diagnosis code
MAX_PROMPT_CHARS	30000	Prompt character budget
FORCE_REVIEW_ON_INPUT_WARNINGS	true	Force review on input warnings
FORCE_REVIEW_ON_FAILED_FINDINGS	true	Force review on failed findings
FORCE_REVIEW_ON_NO_CITATIONS	true	Force review when no citations
FORCE_REVIEW_ON_ZERO_GUIDELINES	true	Force review when no guidelines
ENABLE_AUDIT_REDACTION	false	Enable PII redaction
ENABLE_TAMPER_EVIDENCE	true	Enable SHA256 + DB persistence
AUDIT_LOG_PATH	output/audit_log.jsonl	JSONL audit log path

12.3 Database Setup

Alembic is configured for migrations:

```
alembic upgrade head      # Apply migrations
alembic revision --autogenerate -m "description" # Generate migration
```

Auto-init: The FastAPI lifespan handler calls `init_db()` on startup, which runs `Base.metadata.create_all()`.

12.4 Running the Service

```
# Install dependencies
pip install -r requirements.txt

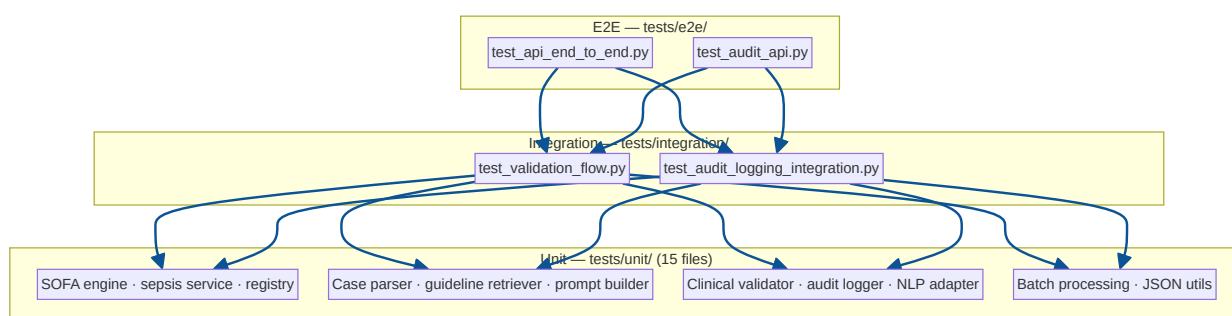
# Set up environment
cp .env.example .env
# Edit .env with credentials

# Run the API
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# API docs
open http://localhost:8000/docs
```

13. Testing Strategy

13.1 Test Pyramid



13.2 Test Markers

```
@pytest.mark.integration # Requires Neo4j + Azure OpenAI
def test_full_validation_flow():
    ...
```

13.3 Mocking Strategy

E2E tests mock the AI layer to avoid live Azure calls:

```
def fake_validate(self, prompt: str, guideline_retrieval_count: int):
    return {
        "model_output": VALID_MODEL_OUTPUT,
        "model_metadata": {...},
        "raw_text": "{...}"
    }
monkeypatch.setattr(ClinicalValidator, "validate", fake_validate)
```

Unit tests mock both Neo4j and Azure OpenAI using `pytest-mock`.

13.4 Running Tests

```
# Unit tests only
pytest tests/unit/ -v

# Integration tests (requires external services)
pytest tests/integration/ -v -m integration

# E2E tests
pytest tests/e2e/ -v

# All tests
pytest -v
```

14. API Reference

14.1 POST /api/v1/validate-case

Request:

```
{
  "case_id": "CASE-2026-CV-19",
  "encounter_id": "ENC-999-001",
  "principal_dx_claimed": "U07.1",
  "secondary_dx": ["J12.82"],
  "procedures": [],
  "poa_flags": {"U07.1": "Y", "J12.82": "Y"},
  "drg_claimed": "177",
  "drg_claimed_description": "Respiratory Infections and Inflammations with MCC",
  "documentation_excerpt": "Patient presented with fever and respiratory distress...",
  "extraction_quality": {
    "completeness_score": 0.98,
    "low_confidence_entities": 0,
    "requires_coder_review": false
  },
  "audit_trail": {
    "source_documents": ["doc_01"],
    "entity_provenance": {"U07.1": "DX-REF-001"}
  }
}
```

Response: Full case payload + `validation_results` block + `_output_path`.

Headers: `X-Request-ID` (optional) — tracked in audit logs.

14.2 POST /api/v1/validate-nlp-extraction

Request: Raw NLP extraction pipeline response (entities, demographics, `claim_clinical_extraction`, vitals, vasopressors).

Response: Converted + validated output.

14.3 POST /api/v1/convert/nlp-extraction

Request: Same as validate-nlp-extraction.

Response: `ClinicalCaseInput` JSON (conversion only, no validation run).

14.4 POST /api/v1/batch/reprocess-samples

Request:

```
{
  "case_ids": ["CASE-1", "CASE-2"], // optional, defaults to all from Postgres
  "max_workers": 8                  // 1-32, default 8
}
```

Response: Summary with per-case results, matched/unmatched counts.

14.5 GET /api/v1/audit/logs

Query Parameters: `case_id`, `validation_status`, `start_date`, `end_date`, `request_id`, `has_error`, `skip`, `limit`, `sort_by`, `sort_order`.

Response: Paginated list of audit log records.

14.6 GET /api/v1/audit/cases/{case_id}/logs

Response: All audit logs for a specific case, most recent first.

15. Configuration & Environment

15.1 Clinical Validation Config

File: `config/clinical_validation.yaml`

Controls which diagnosis-specific validators are active:

```
clinical_validators:
  sepsis:
    enabled: true
    trigger_codes: ["A41", "R65.2"]
    params:
      delta_threshold: 2
  acute_respiratory_failure:
    enabled: false
    trigger_codes: ["J96"]
  malnutrition:
    enabled: false
    trigger_codes: ["E43", "E44"]
```

15.2 Validation Control Flags

Flag	Default	Effect
FORCE_REVIEW_ON_INPUT_WARNINGS	true	Any input warning forces HUMAN_REVIEW
FORCE_REVIEW_ON_FAILED_FINDINGS	true	Any false finding forces HUMAN_REVIEW
FORCE_REVIEW_ON_NO_CITATIONS	true	No AI citations forces HUMAN_REVIEW
FORCE_REVIEW_ON_ZERO_GUIDELINES	true	Zero retrieved guidelines forces HUMAN_REVIEW

These flags make the system **conservative by default** — when in doubt, escalate to human review.

16. Future Enhancements

16.1 Near-Term (Next 3 Months)

Item	Priority	Description
Acute Respiratory Failure validator	High	Implement J96 validator following template pattern
Malnutrition validator	High	Implement E43/E44 validator
CC/MCC validation	High	Procedure validation + complication/comorbidity hierarchy
Pagination for audit API	Medium	Offset/limit pagination for large datasets
Incremental prompt tuning	Medium	Optimize token usage, reduce latency

16.2 Medium-Term (3-6 Months)

Item	Priority	Description
Prometheus metrics	High	Latency, throughput, error rates
Structured logging (JSON)	High	ELK/Loki integration
CC/MCC impact analysis	Medium	Revenue impact of hierarchy changes
Multi-model benchmarking	Medium	GPT-4 vs Claude vs local models
Batch validation optimizations	Medium	Parallelize across multiple Azure regions

16.3 Long-Term (6-12 Months)

Item	Priority	Description
Encoder-only classifier	High	Fast pre-screening to reduce LLM calls
Auto-labeling pipeline	High	Use compliance rules as labeling function
DRG grouping validation	Medium	Validate DRG assignment matches code combinations
CMS audit simulation	Medium	End-to-end compliance stress testing
HIPAA-compliant storage	High	Encrypted at-rest audit logs

Appendix A: File Structure Map

```

clinical-and-coding-validation/
├── app/                                # Thin FastAPI wrapper (all shims)
│   ├── main.py                        # FastAPI app with lifespan, routers
│   └── api/
│       ├── routes.py                  # POST /validate-case, GET /health
│       ├── audit_routes.py            # GET /audit/logs, GET /audit/cases/{id}/logs
│       ├── batch_routes.py            # POST /batch/reprocess-samples
│       └── conversion_routes.py        # POST /convert/nlp-extraction, POST /validate-nlp-extraction
│   ├── config/settings.py             # Shim → SDK
│   ├── schemas/                       # Shims → SDK
│   ├── services/                      # Shims → SDK
│   ├── ai/                            # Shims → SDK
│   ├── retrieval/                     # Shims → SDK
│   ├── prompts/                       # Shims → SDK
│   ├── ingestion/                     # Shim → SDK
│   ├── audit/                         # Shim → SDK
│   ├── compliance/                   # Shim → SDK
│   ├── clinical/                     # Shims → SDK
│   ├── database/                     # Shims → SDK
│   └── utils/                         # Shims → SDK
├── sdk/                               # Core logic (clinical-validation-sdk)
│   ├── pyproject.toml                 # Package definition
│   ├── clinical_validation_sdk/
│   │   ├── __init__.py                # Public exports
│   │   ├── client.py                  # ClinicalValidationClient facade
│   │   ├── config/settings.py          # All env-driven settings
│   │   ├── schemas/
│   │   │   ├── case_schema.py          # ClinicalCaseInput, LabResult, VitalSign
│   │   │   ├── validation_schema.py    # ValidationResults, Citation, ModelMetadata
│   │   │   └── batch_schema.py         # BatchReprocessRequest/Response
│   │   ├── ingestion/case_parser.py    # CaseParser, case_to_prompt_payload
│   │   ├── retrieval/
│   │   │   ├── neo4j_client.py         # Neo4j driver with retry
│   │   │   └── guideline_retriever.py  # Cypher queries, per-code caps
│   │   ├── prompts/
│   │   │   ├── prompt_builder.py       # Template loading, context formatting
│   │   │   └── templates/
│   │   │       └── clinical_validation_prompt.txt
│   │   ├── ai/
│   │   │   ├── azure_openai_client.py # Azure OpenAI with retry
│   │   │   └── clinical_validator.py   # Output structure validation
│   │   ├── clinical/
│   │   │   ├── base.py                 # ClinicalValidatorBase ABC
│   │   │   ├── registry.py             # Gated dispatch
│   │   │   ├── config_loader.py        # YAML loader
│   │   │   ├── sepsis_service.py       # Sepsis-3 decision layer
│   │   │   ├── sofa_engine.py          # 6-component SOFA scoring
│   │   │   └── template_validator.py    # Copy-paste starting point
│   │   ├── compliance/
│   │   │   └── compliance_validator.py  # 8 deterministic rules
│   │   ├── audit/
│   │   │   └── audit_logger.py         # JSONL + PostgreSQL dual-write
│   │   ├── adapters/
│   │   │   └── nlp_extraction_adapter.py # Raw NLP → ClinicalCaseInput
│   │   ├── database/
│   │   │   ├── config.py               # Engine, session factory, init_db
│   │   │   ├── models.py               # AuditLog ORM (UUID PK, JSONB)
│   │   │   ├── schemas.py              # Pydantic DB schemas
│   │   │   ├── repository.py           # DAO with filtering/pagination
│   │   │   └── service.py               # AuditService business logic
│   │   └── utils/
│   │       └── json_utils.py            # safe_json_loads, read/write

```

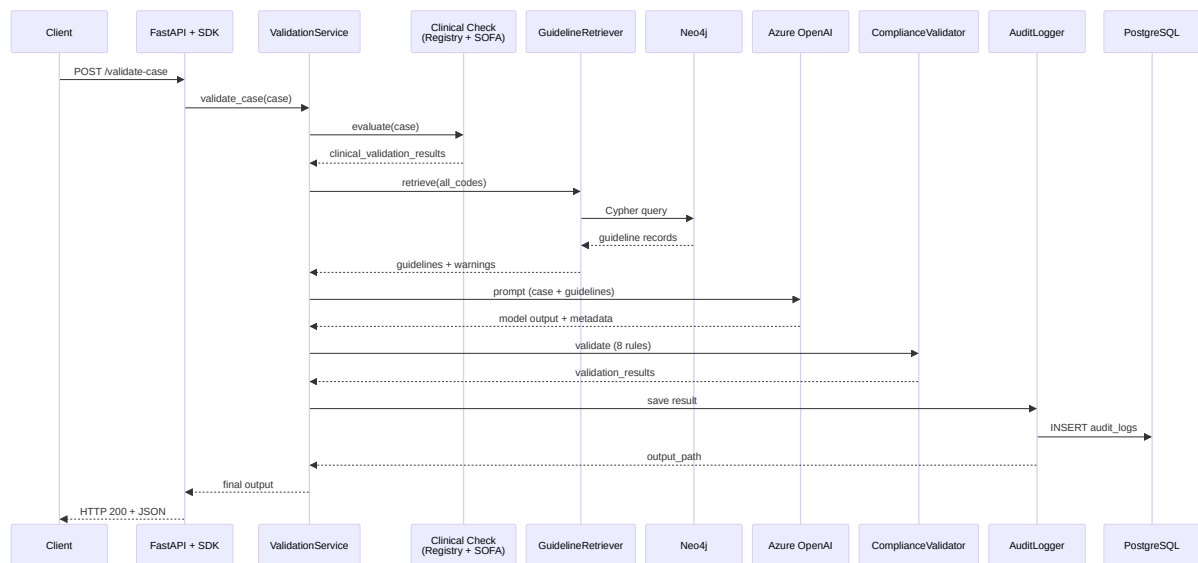
```

| | | logging_utils.py      # Structured logger
| | config/
| | | clinical_validation.yaml  # Gated validator config
| | tests/
| | | unit/                  # 15 unit test files
| | | integration/          # 2 integration test files
| | | e2e/                  # 2 E2E test files
| | samples/                # 9 sample case JSON files
| | output/                 # Validation results + audit log
| | alembic/                # Database migrations
| | scripts/                # Utility scripts
| | engineering/            # Engineering docs & checklists
| | requirements.txt         # App dependencies (references SDK)
| | alembic.ini              # Alembic config
| | pytest.ini               # Pytest markers
| | .env.example             # Environment template
| | .gitignore

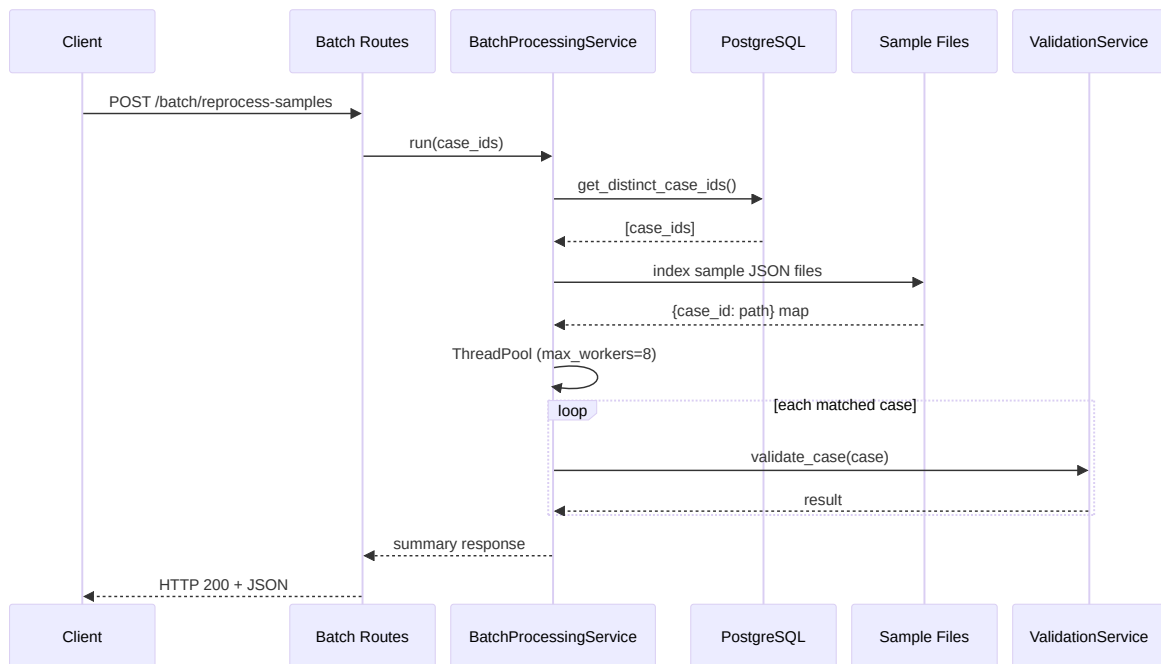
```

Appendix B: Data Flow Diagrams

B.1 Single Case Validation Flow



B.2 Batch Reprocessing Flow



Appendix C: Key Configuration Reference

C.1 Threshold Defaults

Parameter	Default	Description
MIN_COMPLETENESS_SCORE	0.75	Minimum extraction completeness to avoid warnings
MAX_GUIDELINE_CHARS	2000	Truncation limit for individual guideline text
MAX_GUIDELINES_PER_CODE	3	Maximum guideline sections retrieved per code
MAX_PROMPT_CHARS	30000	Total prompt character budget
delta_threshold	2	SOFA increase needed for Sepsis-3 criteria

C.2 Retry Configuration

Service	Attempts	Backoff	Max Wait
Azure OpenAI	3	Exponential	10s
Neo4j queries	3	Exponential	8s

C.3 Database Connection

Setting	Value
Pool class	NullPool
Pool pre-ping	true
Pool recycle	3600s (1 hour)
SQL echo	false